CS 423
Operating System Design: Persistence: RAID, FS API
Apr 9

Ram Kesavan

LOGISTICS

MP3 due by Apr 15, 11:59 CT

Midterm Regrades should be done by early next week

AGENDA / LEARNING OUTCOMES

RAID

- 0, 1, 4, 5

File system API

- Basics of files, directories

RECAP

I/O SCHEDULING IN DEVICE

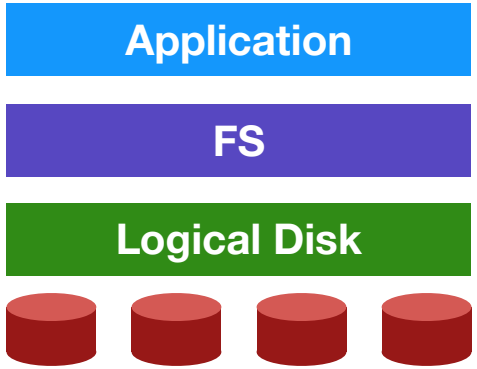
Given a stream of R/W requests, re-order for optimal perf:

- Scheduler in the HDD/SSD firmware
- HDD: optimize for arm movement
- SSD: optimize for FTL wear-leveling

	HDD		SSD	
	Random	Sequential	Random	Sequential
Access	2-15 ms	2 ms	50-150 us	35-500 us
R Throughput	1-3 MB/s	80-160 MB/s	30-500 MB/s	500-550 MB/s
W Throughput	0.5-1 MB/s	80-160 MB/s	0.2-3 GB/s	450-500 MB/s

SOLUTION 2: RAID

Build logical disk from many physical disks.



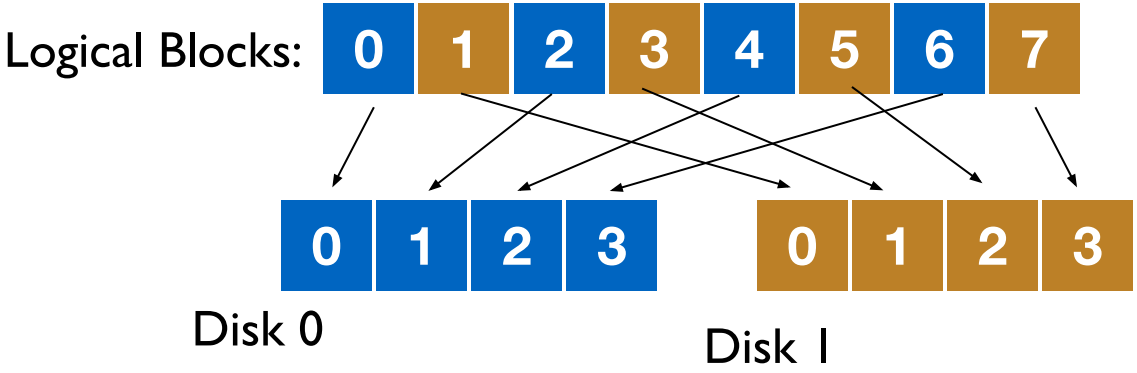
Logical disk gives capacity, performance, reliability

RAID: **R**edundant **A**rray of **I**nexpensive **D**isks

Transparency: no changes to the FS, host, Apps → ease of deployment

RAID-0

Optimize for capacity. No redundancy



RAID 0: STRIPES AND CHUNK SIZE

Chunk size = 1block

	Disk 0	Disk 1	Disk 2	Disk 4
	0	1	2	3
stripe:	4	5	6	7
	8	9	10	11
	12	13	14	15

assume chunk size of 1 for this lecture

Chunk size = 2 blocks

	Disk 0	Disk 1	Disk 2	Disk 4
	0 1	2 3	4 5	6 7
stripe:	8 9	10 11	12 13	14 15

Stripe: blocks/chunks on each disk at the same LBA

RAID-0: ANALYSIS

Capacity: available to file system?

N * C

Reliability: how many failed disks can be tolerated?

0

Latency (random):

D

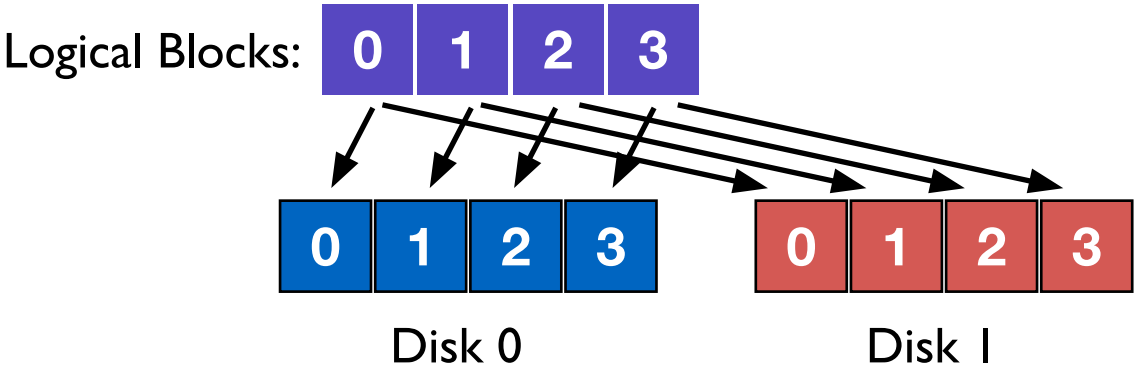
Throughput (sequential, random):

N*S, N*R

More disks improves throughput not latency

N: #disks
C: capacity of 1 disk
S/R: sequential/random throughput of 1 disk
D: latency of one small I/O operation

RAID-1: MIRRORING



Keep two copies of all data.

END RECAP

RAID-1 LAYOUT: MIRRORING

2 disks

Disk 0	Disk 1
0	0
1	1
2	2
3	3

4 disks

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

RAID-1: ANALYSIS

Capacity: available to file system?

Reliability: how many failed disks can be tolerated?

Latency (random):

N: #disks
C: capacity of 1 disk
S/R: sequential/random throughput of 1 disk
D: latency of one small I/O operation

Disk 0	Disk 1
0	0
1	1
2	2
3	3

RAID-1: THROUGHPUT

What is steady-state throughput for

- random reads?

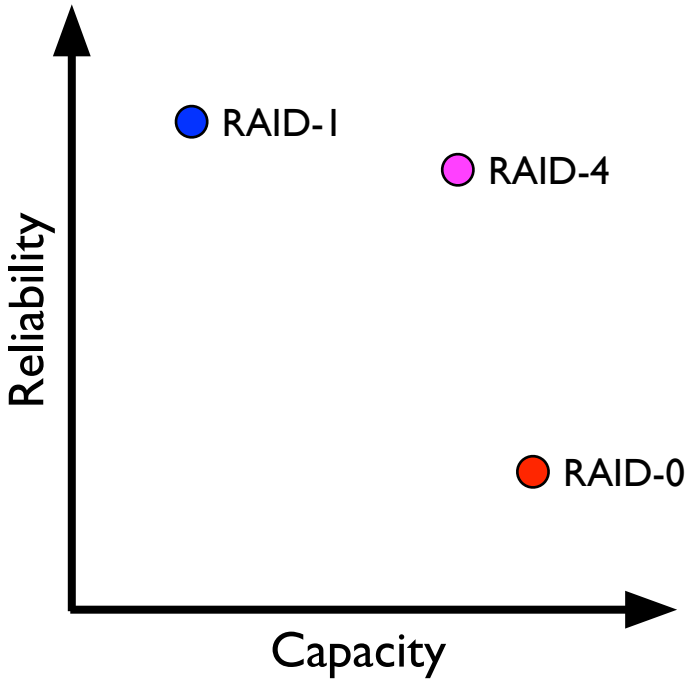
- random writes?

- sequential writes?

- sequential reads?

N: #disks
C: capacity of 1 disk
S/R: sequential/random throughput of 1 disk
D: latency of one small I/O operation

Disk 0	Disk 1	Disk 2	Disk 4
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7


RAID-1
RAID-4
RAID-0
Reliability
Capacity

RAID-4 STRATEGY

Concept of a **parity** disk

- Each **RAID group** has n data disks + 1 parity disk

Build a math equation across blocks in a stripe

- Each stripe has n + 1 blocks (assume chunk size = 1)
- Equation has n + 1 variables
- If any n are known, can solve for the missing unknown

The math: ideally cheap to compute, parallelizable

Block on a *bad* disk would be the missing unknown

RAID 4: EXAMPLE

RAID Group: (4,1)
4 data & 1 parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

What function to compute parity?

Let's assume a block is just a single bit for now!

RAID-4: ANALYSIS

Capacity: available to file system?

Reliability: how many failed disks can be tolerated?

Latency (random read):

Latency (random write):

N: #disks
C: capacity of 1 disk
S/R: sequential/random throughput of 1 disk
D: latency of one small I/O operation

RAID-4: THROUGHPUT

Steady-state throughput for

- sequential reads?
- sequential writes?
- random reads?
- random writes?

Important: I/O cost is >> parity computation cost

PARITY MATH: ADDITIVE VS SUBTRACTIVE

Write a new C1

C0	C1	C2	C3	P
0	0	1	1	XOR(0,0,1,1)

Additive:

P_{new} = XOR($C0_{old}$,$C1_{new}$,$C2_{old}$,$C3_{old}$)

Subtractive:

P_{new} = XOR($C1_{old}$,$C1_{new}$,P_{old})

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

RAID-5

Disk0	Disk1	Disk2	Disk3	Disk4
-	-	-	-	P
-	-	-	P	-
-	-	P	-	-

...

Parity disk is the bottleneck: rotate parity across the disks

Parity block location ~= stripe-number modulo n

RAID-5: ANALYSIS

Capacity: available to file system?

Reliability: how many failed disks can be tolerated?

Latency (random read):

Latency (random write):

N: #disks
C: capacity of 1 disk
S/R: sequential/random throughput of 1 disk
D: latency of one small I/O operation

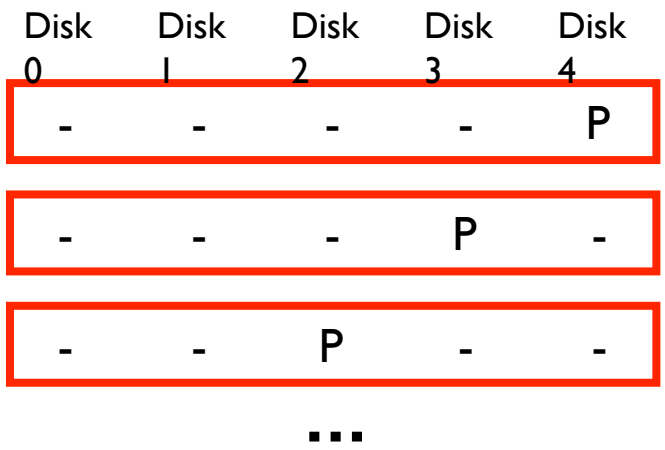
Disk0	Disk1	Disk2	Disk3	Disk4
-	-	-	-	P
-	-	-	P	-
-	-	P	-	-

...

RAID-5: THROUGHPUT

What is steady-state throughput for RAID-5?

- sequential reads?
- sequential writes?
- random reads?
- random writes?



RAID LEVEL COMPARISONS

	Reliability	Capacity	Read latency	Write Latency	Seq Read	Seq Write	Rand Read	Rand Write
RAID-0	0	C*N	D	D	N * S	N * S	N * R	N * R
RAID-1	1	C*N/2	D	D	N/2 * S	N/2 * S	N * R	N/2 * R
RAID-4	1	(N-1) * C	D	2D	(N-1)*S	(N-1)*S	(N-1)*R	R/2
RAID-5	1	(N-1) * C	D	2D	(N-1)*S	(N-1)*S	N * R	N/4 * R

RAID 4 & 5: RECONSTRUCTION

Failed disk needs to be reconstructed

- A spare disk is used to replace the failed disk
- Parity is used to reconstruct each block in that disk

In the meantime, what happens to reads/writes to that disk?

How fast to run reconstruction?

- Too slow → ?
- Too fast → ?

RAID: SUMMARY

RAID: a faster, larger, more reliable disk system

- Used with HDDs or SSDs
- Typically, all disks are similar & of the same size
- Other levels: RAID 10, RAID 01
- RAID 2, RAID 3 (ignore)
- RAID-6, DP (double parity), TP (triple parity)

Single block numbering built over many physical storage devices

Different mapping and redundancy schemes

Different trade-offs

NEXT

Thus far, bottom-up stuff:

- Storage devices
- Storage media
- RAID

Next, top-down stuff:

- Files and File system API
- Different trade-offs

After that: file system

Files & File System

File: sequence of persisted bytes

- sections of a file may be “holes”

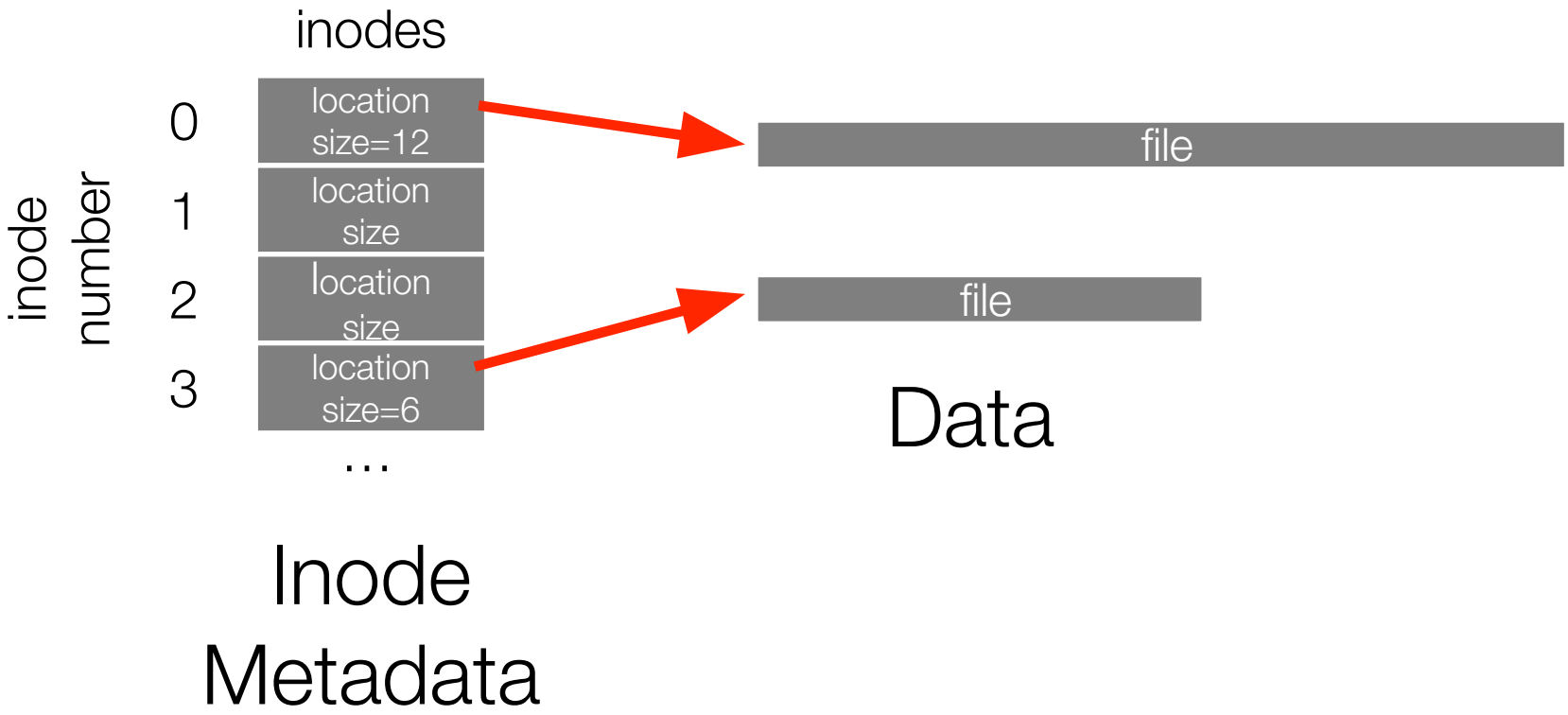
File system refers to both:

- a collection of files (mounted/unmounted together)
- module of the OS that manages this collection of files

Name/address for a file:

- unique id (within the file system): inode number aka inum
- path
- file descriptor

Inodes & Files



File API (attempt 1)

```
read(int inode, void *buf, size_t nbyte)

write(int inode, void *buf, size_t nbyte)

seek(int inode, off_t offset)
```

Disadvantages?

File Path Name

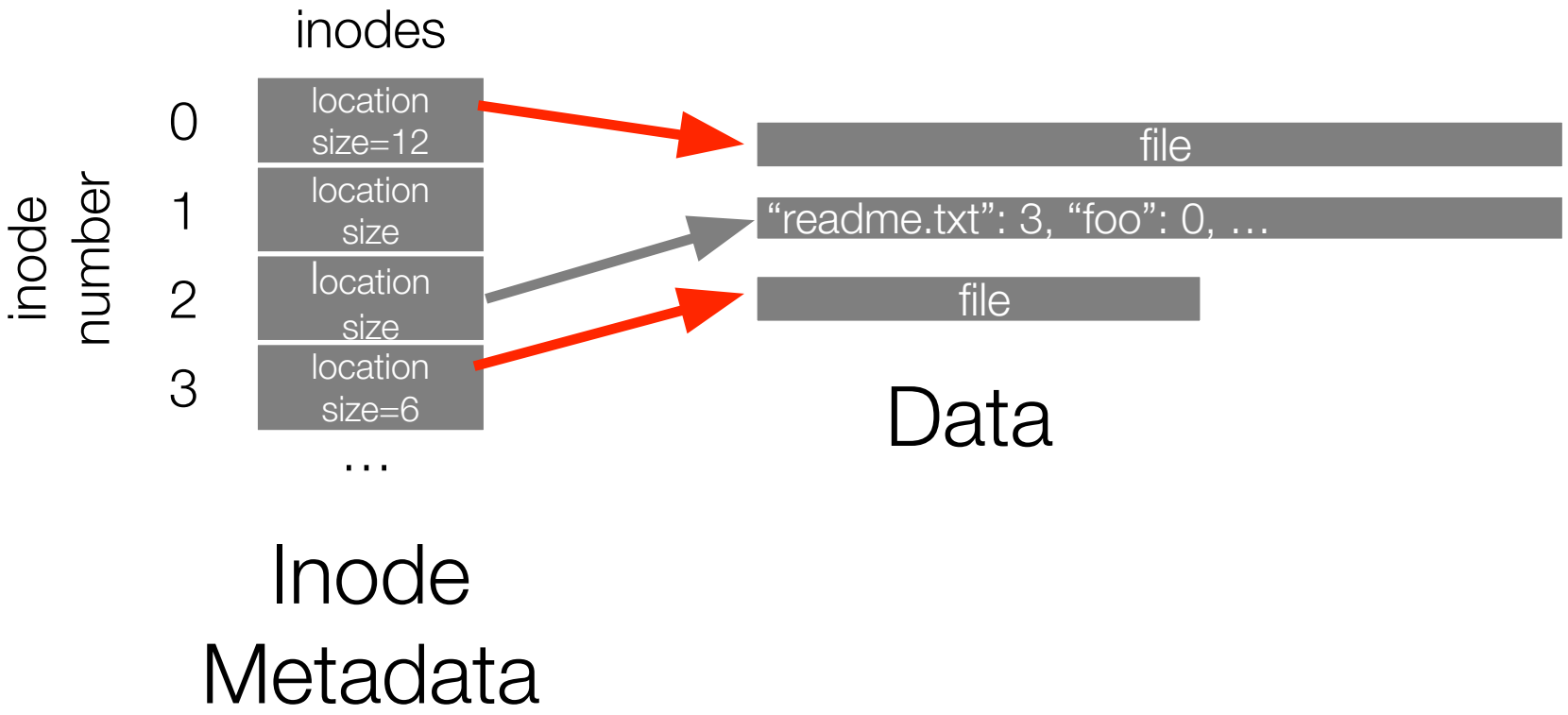
Humans prefer to:

- Use string names instead of inums
- Organize files into hierarchical structures

The file system internally uses only inums

It stores *path-to-inode* mappings in special inodes called *directories (or folders).*

A directory inode



Paths

Directory Tree instead of single root directory

File name needs to be unique within a directory

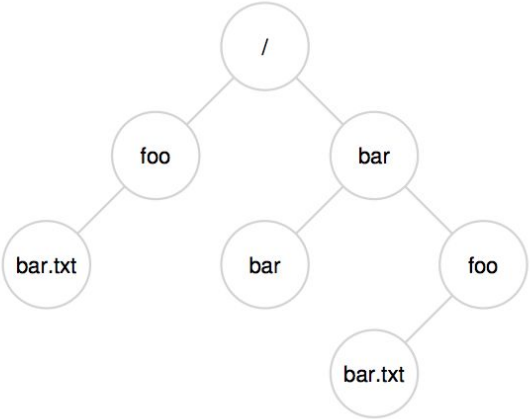
/usr/lib/file.so

/tmp/file.so

Directory stores filename-to-inode mapping

"Directory traversal" needed to access a file

absolute vs relative pathname



File API (attempt 2)

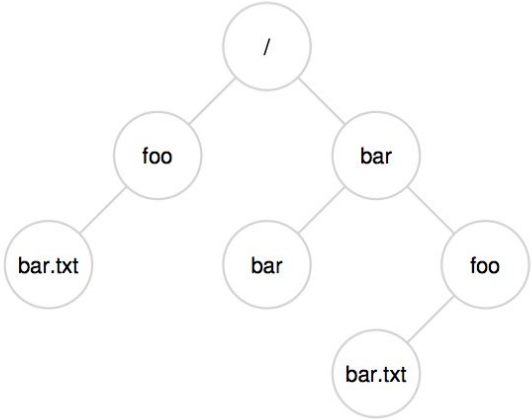
```
read(char *path, void *buf, off_t offset, size_t nbyte)

write(char *path, void *buf, off_t offset, size_t nbyte)
```

Disadvantage?

Traversal is expensive: /bar/foo/bar.txt

load "/" inode (a fixed inode), search for "bar"
load "bar" inode, search for "foo"
load "foo" inode, search for "bar.txt"
load "bar.txt" inode

Goal: traverse once



File Descriptor (fd)

Solution:

- Expensive traversal done once (to **open** file)
- The open API returns a file descriptor (fd), which is an int
- All subsequent API calls (reads/writes/etc) use fd
  - An opaque handle that hides the inum within it, and current offset

Each process has a file descriptor table:

- maps each fd (handed out) to a pointer into a system-wide file descriptor table

Special file descriptors: 0 (stdin), 1 (stdout), 2 (stderr)

- UNIX hands out file descriptors starting from 3

File API (attempt 3)

```
int fd = open(char *path, int flag, mode_t mode)

read(int fd, void *buf, size_t nbyte)

write(int fd, void *buf, size_t nbyte)

close(int fd)
```

advantages:

- string names
- hierarchical
- traverse once
- offsets precisely defined

FD & offsets

System Calls	Return Code	Current Offset
`fd = open("file", O_RDONLY);`	3	0
`read(fd, buffer, 100);`	100	100
`read(fd, buffer, 100);`	100	200
`read(fd, buffer, 100);`	100	300
`read(fd, buffer, 100);`	0	300
`close(fd);`	0	–

OFT: Open File Table to track all open files

FD Offsets

System Calls	Return Code	OFT[10] Current Offset	OFT[11] Current Offset
`fd1 = open("file", O_RDONLY);`	3	0	–
`fd2 = open("file", O_RDONLY);`	4	0	0
`read(fd1, buffer1, 100);`	100	100	0
`read(fd2, buffer2, 100);`	100	100	100
`close(fd1);`	0	–	100
`close(fd2);`	0	–	–

LSEEK and READ

```
off_t lseek(int fd, off_t offset, int whence)
```

If whence is **SEEK_SET**, the offset is set to offset bytes.
If whence is **SEEK_CUR**, the offset is set to its current location plus offset bytes.
If whence is **SEEK_END**, the offset is set to the size of the file plus offset bytes.

Assume head is on track 1
Suppose we do lseek to X and the sector for X is on track 4
Where is head immediately after lseek?

Shared Entries in OFT

Fork:

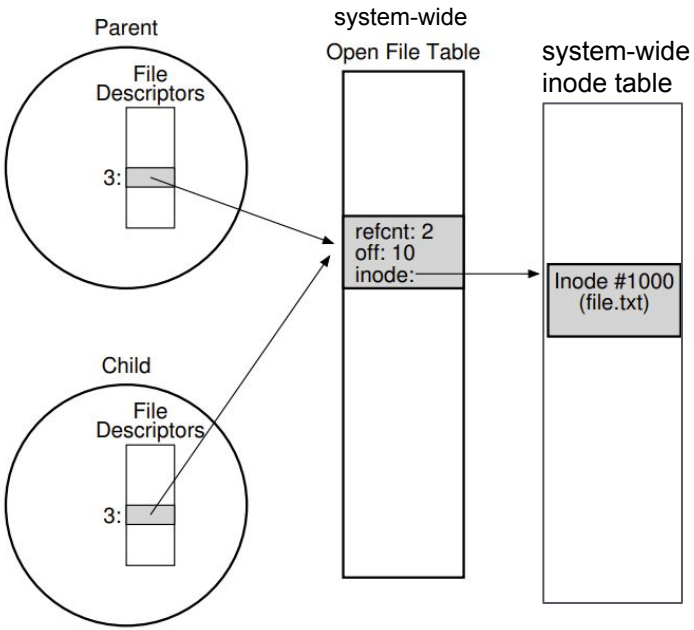
```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
               (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

What offset does the parent print?

What offset does the child print?

FD-Table→OFT→Inum Table

What's happening here?



Child gets its own FD-table, which is a copy of the parent's

- Child can independently close files
- But, child & parent share offset

System-wide OFT

- Per-process FD-table points into it
- Each OFT entry has its own offset

System-wide inode table

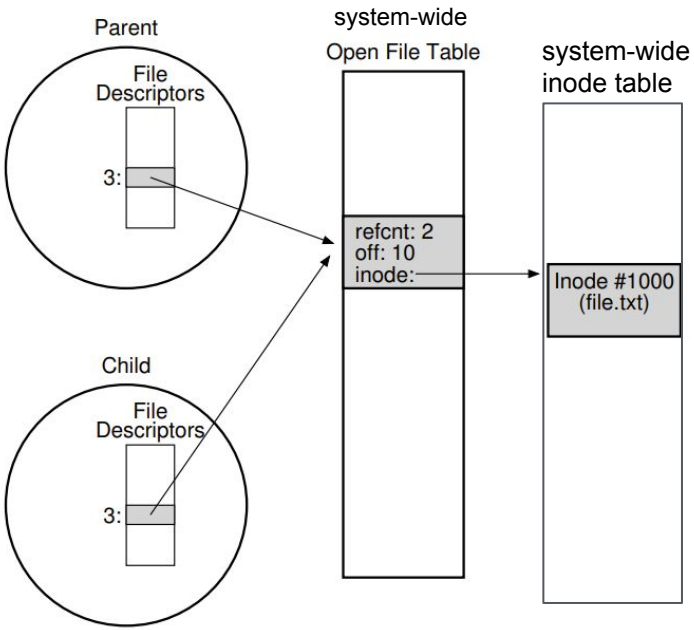
- OFT entries point into it

Many FD table entries may point to one OFT entry
Many OFT entries may point to one inode

OFT & Inode-table are per- file system

FD-Table→OFT→Inum Table

What's happening here?



In Linux kernel, the per-process file descriptor rable is located in a nested location.

In the file `include/linux/sched.h`

Start from `task_struct`

→ `struct files_struct *files;`

→ `struct fdtable *fdt;`

→ `struct file fd[ ];`

DUP System Call

```
int fd1 = open("file.txt"); // returns 3
read(fd1, buf, 12);
int fd2 = open("file.txt"); // returns 4
int fd3 = dup(fd2);         // returns 5; points to the same OFT entry
```

Fsync

File system holds recently written data in memory for a bit

Write buffering improves performance. Why?

But what if system crashes before buffers are flushed?

fsync(int fd) forces buffers to flush to disk

Guarantees that data has been persisted durably

Paths and Links

(Hard) Link:

```
% ln <filename> <linkname>
```

Each inode has a field called “nlinks”

When is it incremented?

When is it decremented?

Deleting Files

Use the `unlink()` system call for deleting files

- Results in decrement of `nlinks` in inode
- if `nlinks == 0`
  - inode (and associated file) is **garbage collected**
  - File in directory is deleted

Actually, the garbage collection happens only after the last fd pointing to that file has called `close(fd)`

Rename

rename(char *old, char *new):

Do we need to copy/move data?

How does the FS implement this?

Does it matter whether the old and new names are in the same directory or different directories?

Rename

```
rename(char *old, char *new):
```

- effectively unlinks the old file
- creates a link for the new file

Only changes name of file, does not move data

Even when renaming to new directory

What can go wrong if system crashes at wrong time?

Atomic File System Update

How do we make atomic updates to the file system?

Eg. Atomically move file.txt from directory foo to directory bar

If system crashes:

file system should accurately have that file only in either foo or in bar

not both & not neither

Next

File system implementation

A simple ext2 like FS

Then, fsck, journaling, FFS, …